

Computer Network Practice: IPC

CC Lab.

Taewook Kim (TA)

Computer Communication Lab.

Dept. of Computer Engineering,

Chungbuk National University





Lecture Plan

Week	Date	Contents
1	9.1	-
2	9.8	Introduction & Ping
3	9.15	Inter-Process Communication
4	9.22	File Transfer 1
5	9.29	File Transfer 2
6	10.6	ARP
7	10.13	GARP, RARP
8	10.20	(middle test)
9	10.27	Internet Protocol Routing I & Static Router
10	11.3	Internet Protocol Routing II & Static Router
11	11.10	Internet Protocol Routing III & Static Router
12	11.17	Network Address Translation I
13	11.24	Network Address Translation II
14	12.1	Network Address Translation III
15	12.8	(final test)





Contents

- ◆ Background
 - ❖ Protocol Stack
 - ❖ Encapsulation
 - ❖ Demultiplexing
 - ❖ Protocol Formats
- ◆ Inter-Process Communication
 - ❖ Assignment Notice
- ◆ Q&A



Background

CLab.



Protocol Stack

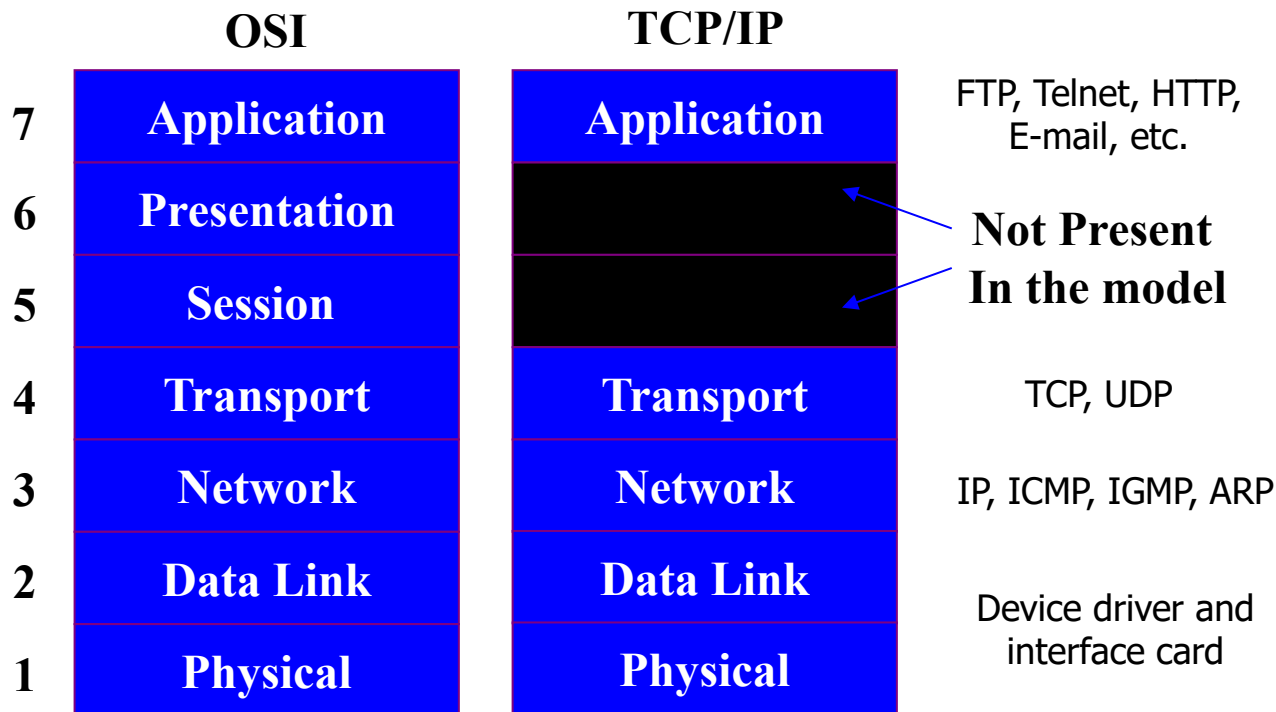


Figure 1_1. The OSI Reference Model & The TCP/IP Reference Model





Encapsulation

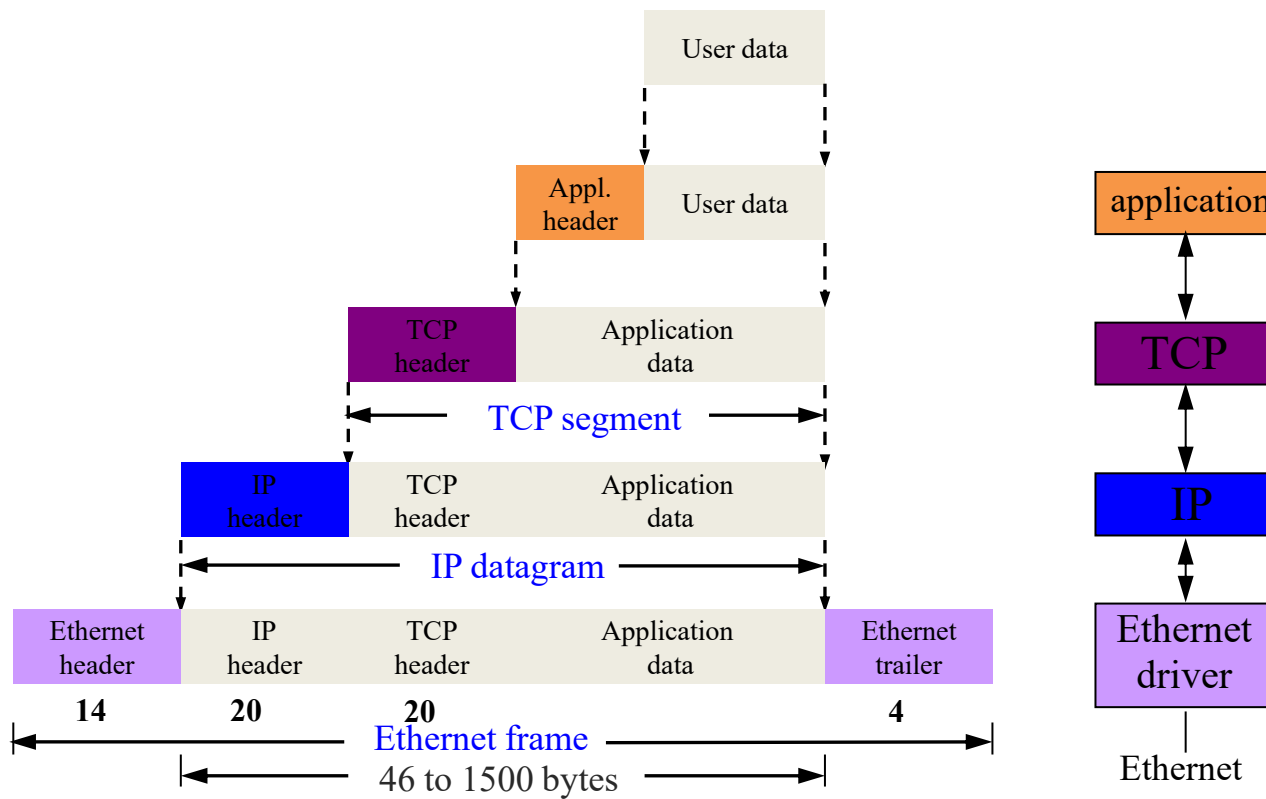


Figure 2. Encapsulation of data as it goes down the protocol stack



Demultiplexing

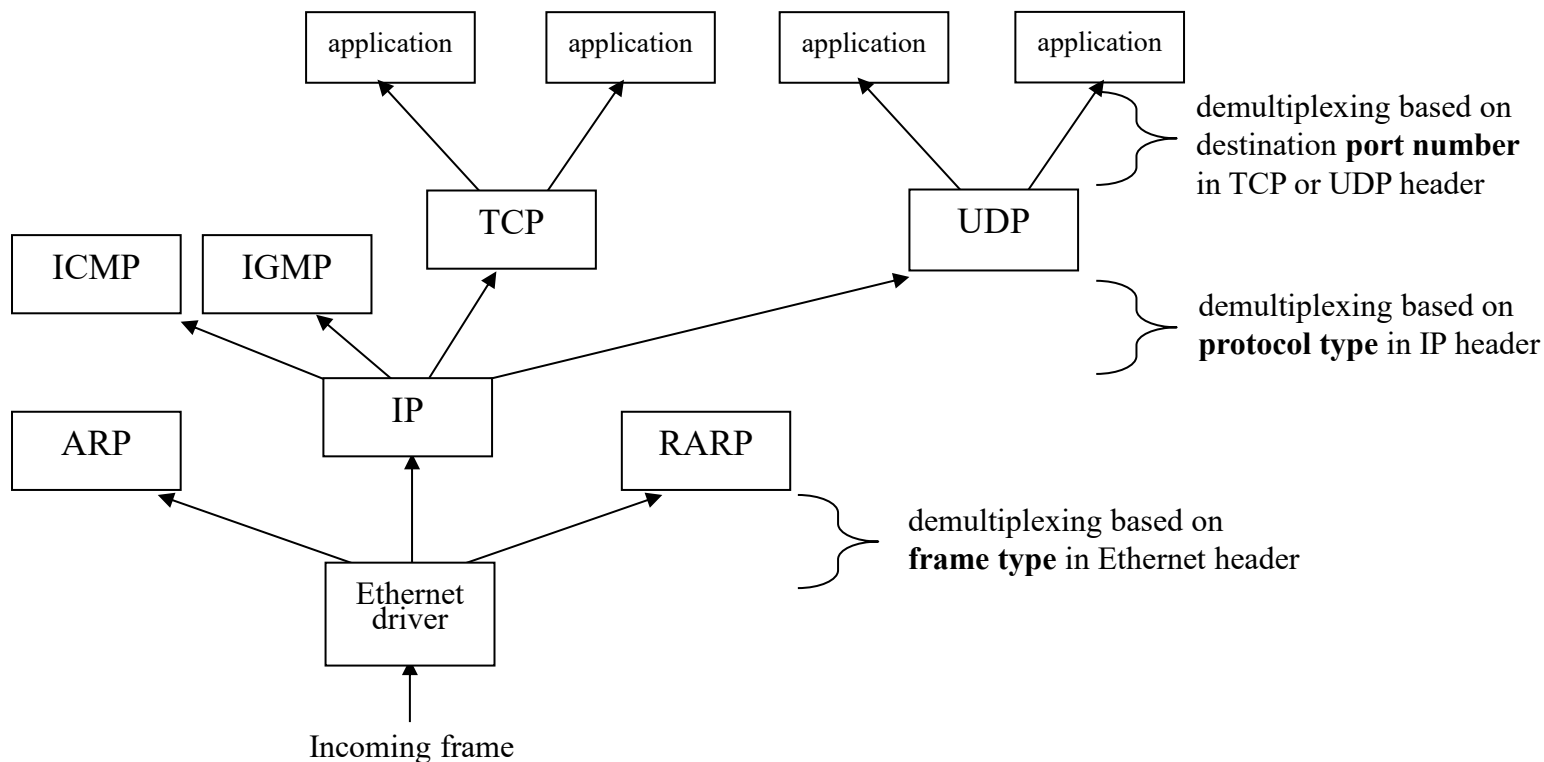


Figure 3. The demultiplexing of a received Ethernet frame





Protocol Formats

◆ Ethernet

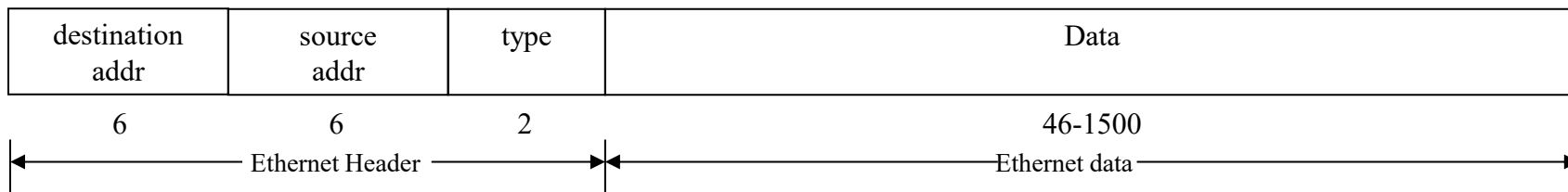


Figure 4. Ethernet encapsulation (RFC 894)

◆ Implementation of protocol stack using C++ with Ethernet

```
struct _ETHERNET_HEADER {  
    unsigned char    enet_dstaddr[ 6 ];  
    unsigned char    enet_srcaddr[ 6 ];  
    unsigned short   enet_type    ;  
    unsigned char    enet_data[ 1500 ];  
}
```





Protocol Formats (cont'd)



Ethernet

```
5  #if !defined(AFX_ETHERNETLAYER_H__7857C9C2_B459_4DC8_B9B3_4E6C8B587B29__INCLUDED_)
6  #define AFX_ETHERNETLAYER_H__7857C9C2_B459_4DC8_B9B3_4E6C8B587B29__INCLUDED_
7
8  #if _MSC_VER > 1000
9  #pragma once
10 #endif // _MSC_VER > 1000
11
12 #include "BaseLayer.h"
13 #include "pch.h"
14
15 class CEthernetLayer
16 | : public CBaseLayer
17 {
18 private:
19 | inline void      ResetHeader();
20
21 public:
22 | BOOL            Receive(unsigned char* ppayload);
23 | BOOL            Send(unsigned char* ppayload, int nlength);
24 | void            SetDestinAddress(unsigned char* pAddress);
25 | void            SetSourceAddress(unsigned char* pAddress);
26 | unsigned char*  GetDestinAddress();
27 | unsigned char*  GetSourceAddress();
28
29 | CEthernetLayer(char* pName);
30 | virtual ~CEthernetLayer();
31
32 | typedef struct _ETHERNET_HEADER {
33 |
34 |     unsigned char    enet_dstaddr[6];        // destination address of ethernet layer
35 |     unsigned char    enet_srcaddr[6];        // source address of ethernet layer
36 |     unsigned short    enet_type;             // type of ethernet layer
37 |     unsigned char     enet_data[ETHER_MAX_DATA_SIZE]; // frame data
38 |
39 | } ETHERNET_HEADER, * PETHERNET_HEADER;
40
41 protected:
42 | ETHERNET_HEADER m_sHeader;
43 };
44
45 #endif // !defined(AFX_ETHERNETLAYER_H__7857C9C2_B459_4DC8_B9B3_4E6C8B587B29__INCLUDED_)
```





Protocol Formats (cont'd)

◆ Address Resolution Protocol

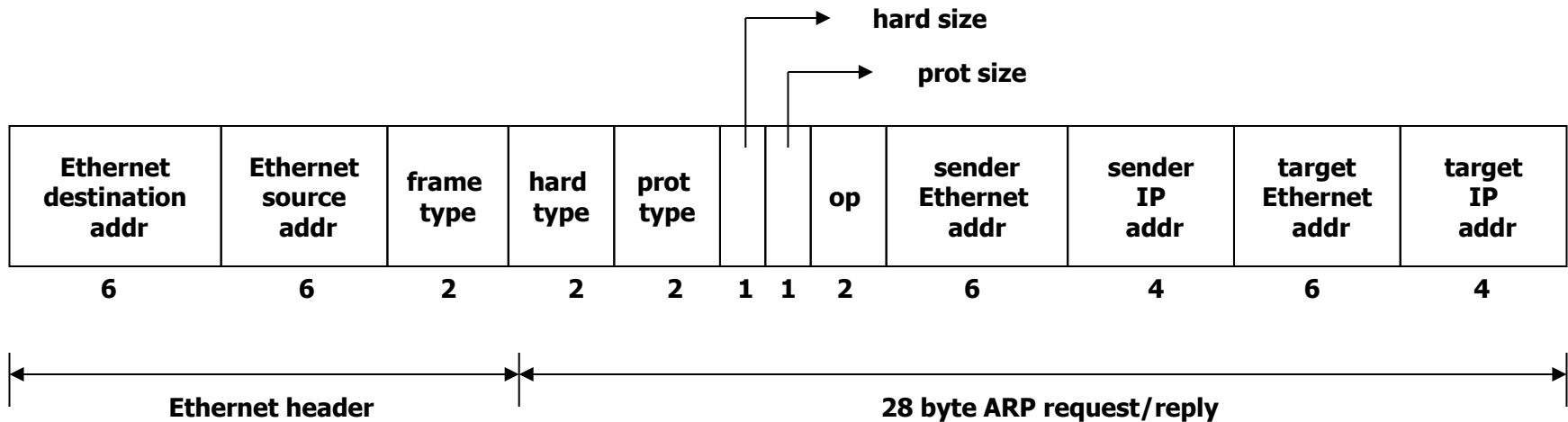


Figure 6. Format of ARP request or reply packet when used on an Ethernet





Protocol Formats (cont'd)

◆ Internet Protocol

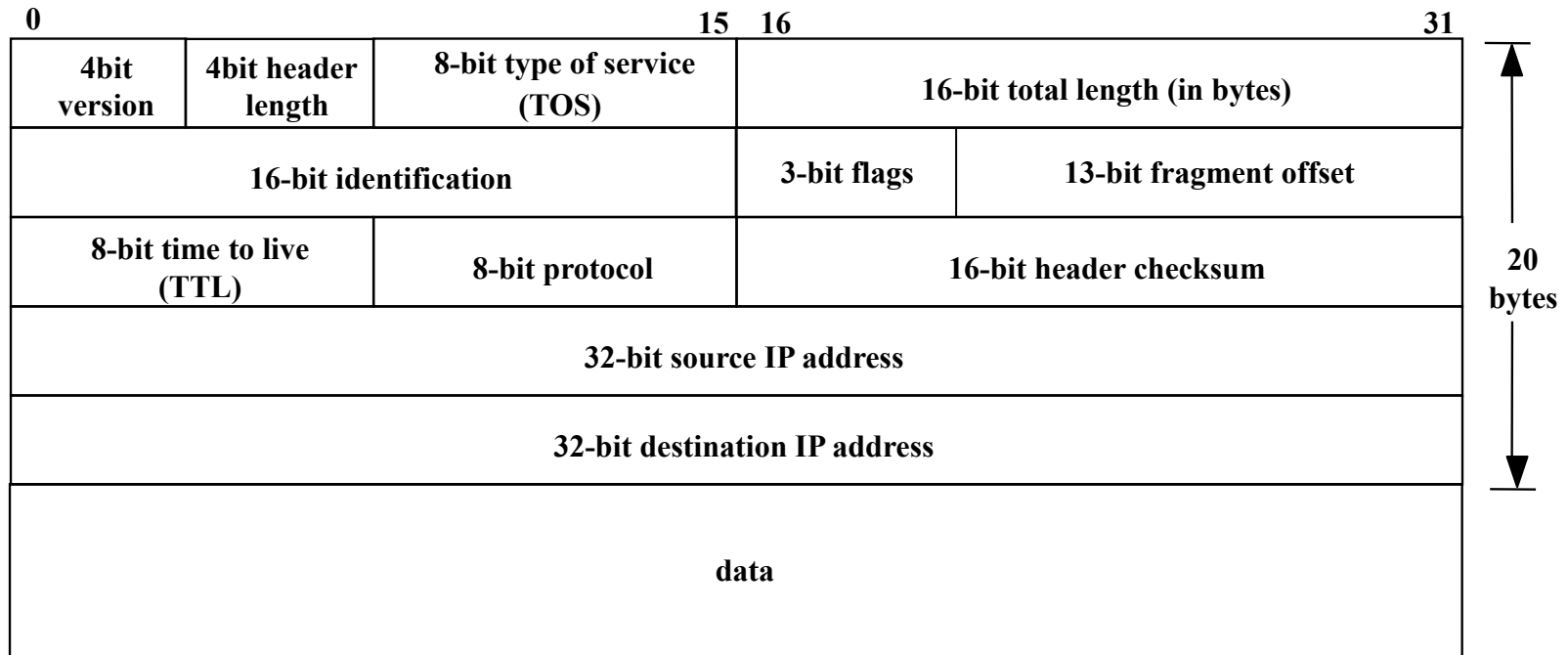


Figure 5. IP datagram, showing the fields in the IP header



Inter-Process Communication

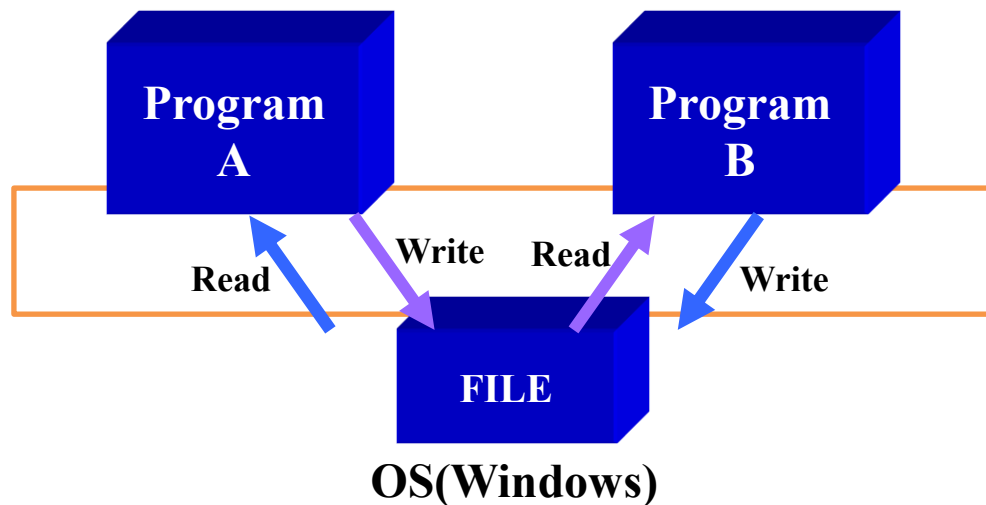
C Lab.



Inter-Process Communication

◆ Background

- ❖ OS의 커널에서 임의의 프로세스가 다른 프로세스의 메모리 주소 공간에 접근할 수 없도록 함
- ❖ 하지만 경우에 따라 프로세스 간 데이터를 공유해야 하는 경우가 발생함





Inter-Process Communication

◆ Problem

- ❖ 하나의 파일을 다중 프로세스들이 동시에 사용하고자 할 때
그 데이터를 필요로 하는 프로세스에게 제대로 전달이 안될 수 있음
다시 말해서, 프로세스들 간 동기화 문제가 발생

◆ Solution

- ❖ IPC (Inter-process Communication) 사용

◆ 대부분 OS

- File
- Signal
- Socket
- Message Queue

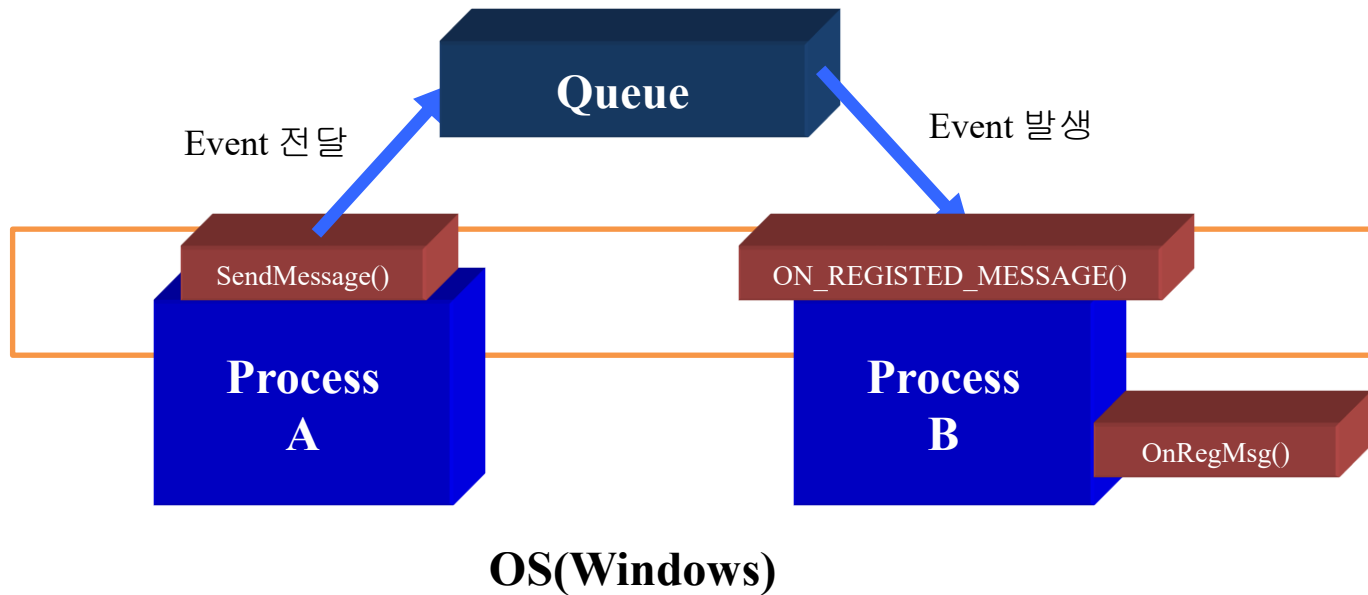
◆ POSIX OS

- Pipe
- Shared Memory
- Semaphore



Inter-Process Communication

- ◆ Message Queue
 - ❖ Register Using C++





Assignment #03

◆ 과제 개요

- ❖ Process A의 사용자로부터 입력 받은 메시지를 Process B의 메시지 창에 출력

◆ 제출 파일

- ❖ 프로젝트 압축 파일
- ❖ 보고서

◆ 실습 개요

- 실습 일시 및 장소
- 실습 목적

◆ 프로토콜 스택

- 구조 설명 (프로토콜의 역할 등)

◆ 구현 설명

- 코드 설명 (ipc.cpp, pch.cpp, framework.h, ipc.h, pch.h, resource.h, targetver.h 제외)

◆ 실행 결과

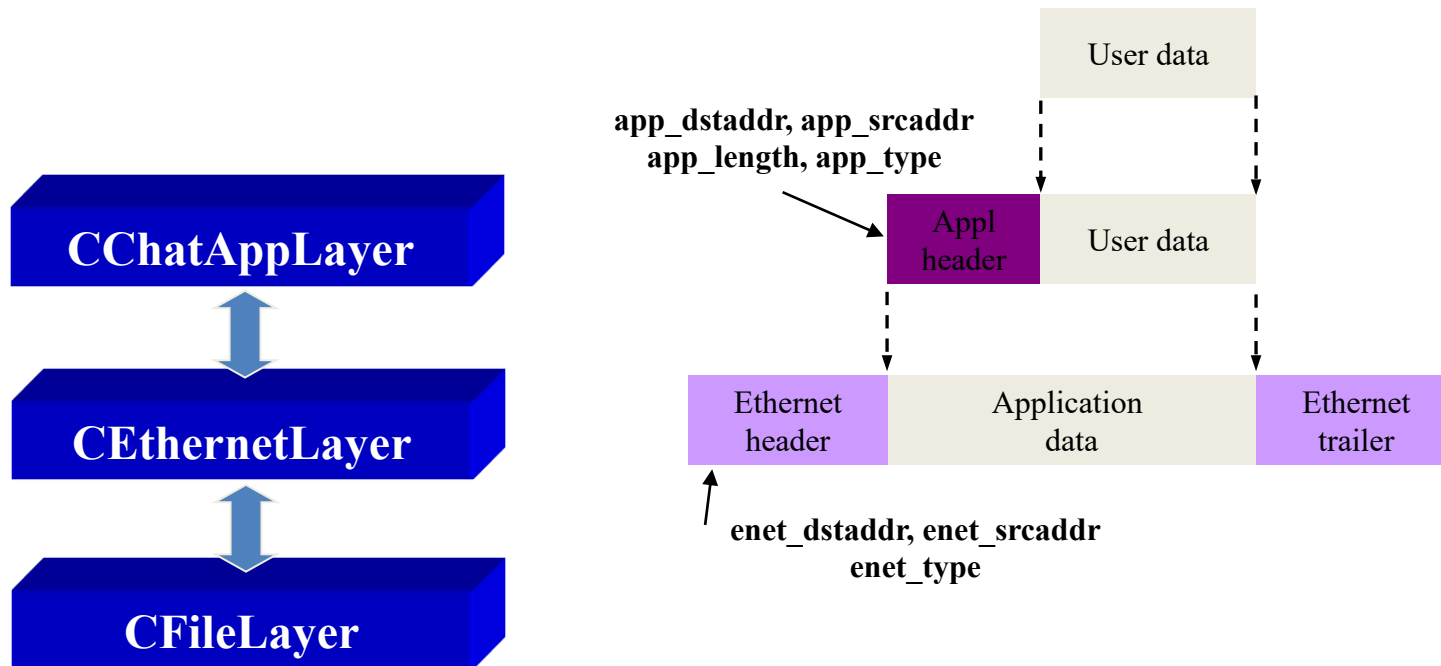
- 프로그램 결과 화면





Assignment #03

◆ Hierarchical Architecture





Assignment #03

◆ 참고 사항

- ❖ 내 Application과 상대 Application 주소 설정
 - ◆ 각 프로그램은 고유한 프로그램 ID를 가지고 있다.
- ❖ Process ID의 세부 사항
 - ◆ process ID는 32bits의 크기(unsigned integer)를 가지고 있다.
- ❖ 메시지 전송을 레이어 상의 메시지형식은 다음과 같다.
 - ◆ source ID(4bytes), destination ID(4bytes), message length(2bytes), message type(1byte), message data(# bytes)
- ❖ 메시지 수신
 - ◆ 메시지는 상대방이 자기주소로 보낸 메시지와 broadcast message 만을 수신하며 나머지는 discard한다.

◆ 실습 환경

- ❖ Windows OS
- ❖ Visual Studio 2026
 - ◆ 최신 v145 빌드 도구용 C++ MFC(x86 및 x64)





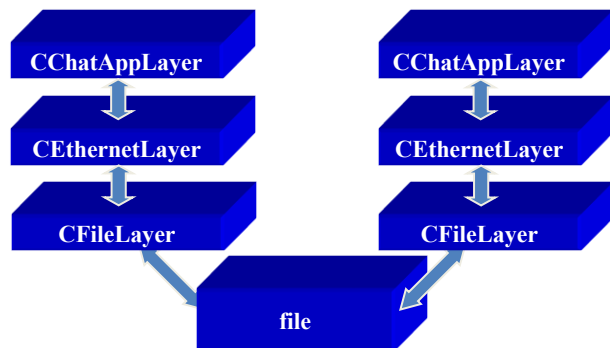
Assignment #03

◆ 참고 사항

❖ Protocol Stack을 사용한 메시지 송신

◆ CChatAppLayer → CEthernetLayer → CFileLayer

- CChatAppLayer:
CChatApp header에 data(메시지)를 붙여서 Ethernet Layer에게 전체를 전달
- CEthernetLayer:
CChatAppLayer로부터 전달 받은 정보를 Ethernet의 data에 넣어서 CFileLayer에게 전체를 전송
- CFileLayer:
CEthernetLayer로 전송 받은 데이터를 파일로 기록





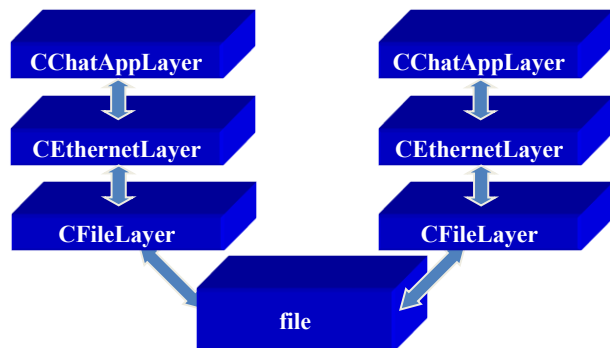
Assignment #03

◆ 참고 사항

❖ Protocol Stack을 사용한 메시지 수신

◆ CChatAppLayer <- CEthernetLayer <- CFileLayer

- CFileLayer:
File로부터 동기화를 통해 전달된 메시지를 받은 후
해당 파일의 데이터를 읽어서 Ethernet으로 데이터를 전달
- CEthernetLayer:
File Layer로부터 받은 데이터 중에서 Ethernet header를 분리한 후
Ethernet data 부분을 CChatAppLayer로 전달
- CChatAppLayer:
CEthernetLayer로부터 받은 데이터 중에서 CChatApp header를 분리한 후
CChatApp data를 화면에 보여줌





Assignment #03

◆ 실행 예제

❖ 송신측

- ◆ 송신측은 파일에 기록을 완료하고, 기록한 것을 수신측에게 알림
- ◆ 수신측으로부터 응답 메시지가 올 때까지 **2초간** 대기
 - 응답이 없으면, 화면에 time-out 메시지를 출력

❖ 수신측

- ◆ 송신측으로부터 알림을 받고 CFileLayer를 통하여 파일을 읽음
- ◆ 정상적으로 파일을 읽어 화면에 출력하였으면,
송신측에게 정상적으로 동작했다는 메시지 응답





Assignment #03

◆ 실행 예제

❖ 프로그램 화면에 나타나야 할 정보를 정의한다.

◆ Source / Destination Address, Message etc.

❖ 메시지 출력 방식

◆ 1번 process에서 2번 process로 정상적인 메시지 전송이 이루어진 경우 : [1:2]입력메시지

Ex.) 안녕하세요. -> “[1:2] 안녕하세요.”

◆ 1번에서 **broadcasting** 한 메시지일 경우 :

Ex.) 안녕하세요. -> “[1:BROADCAST] 안녕하세요.”

◆ **Broadcast Message** 경우에는 응답메시지가 하나만 도착해도 올바른 수신이 된 것으로 인식

❖ 응답이 없는 경우 : 메시지 출력 후 “time-out” message를 다시 출력

◆ Ex.) 안녕하세요. -> “[1:2]안녕하세요.” 출력 후 “time-out”을 다시 출력





Assignment #03

◆ 제출일

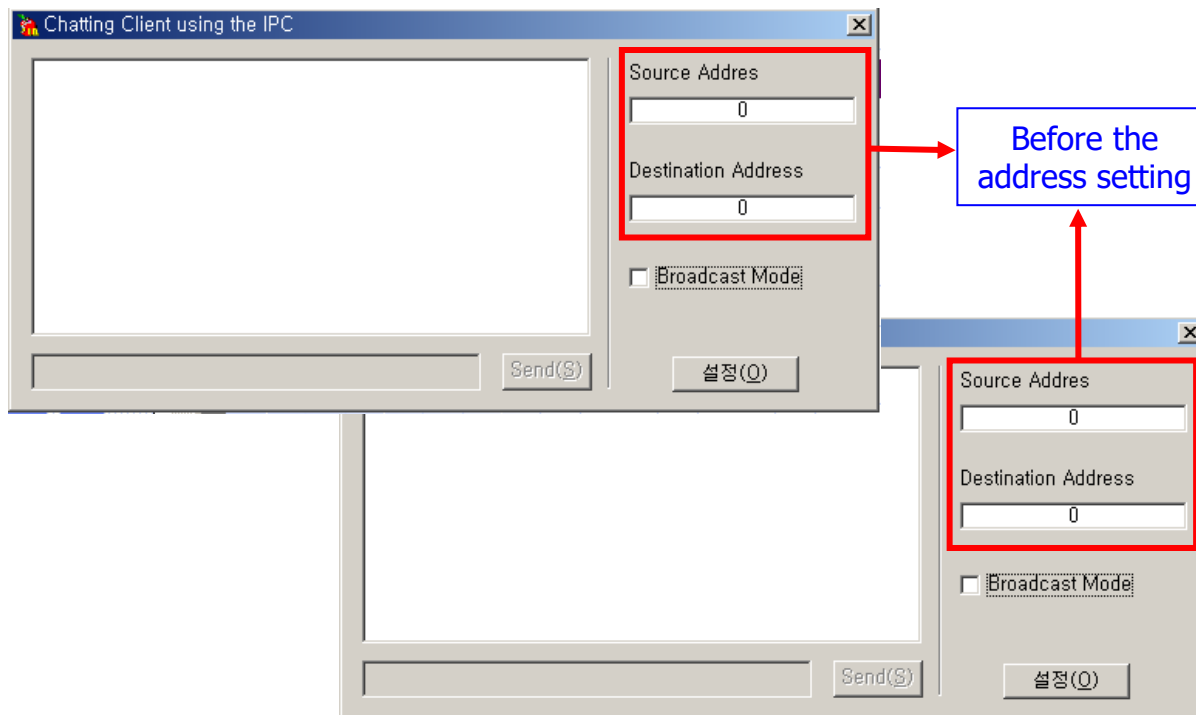
- ❖ 9월 13일 11:59분까지
- ❖ 개인 별 제출





User Interface example

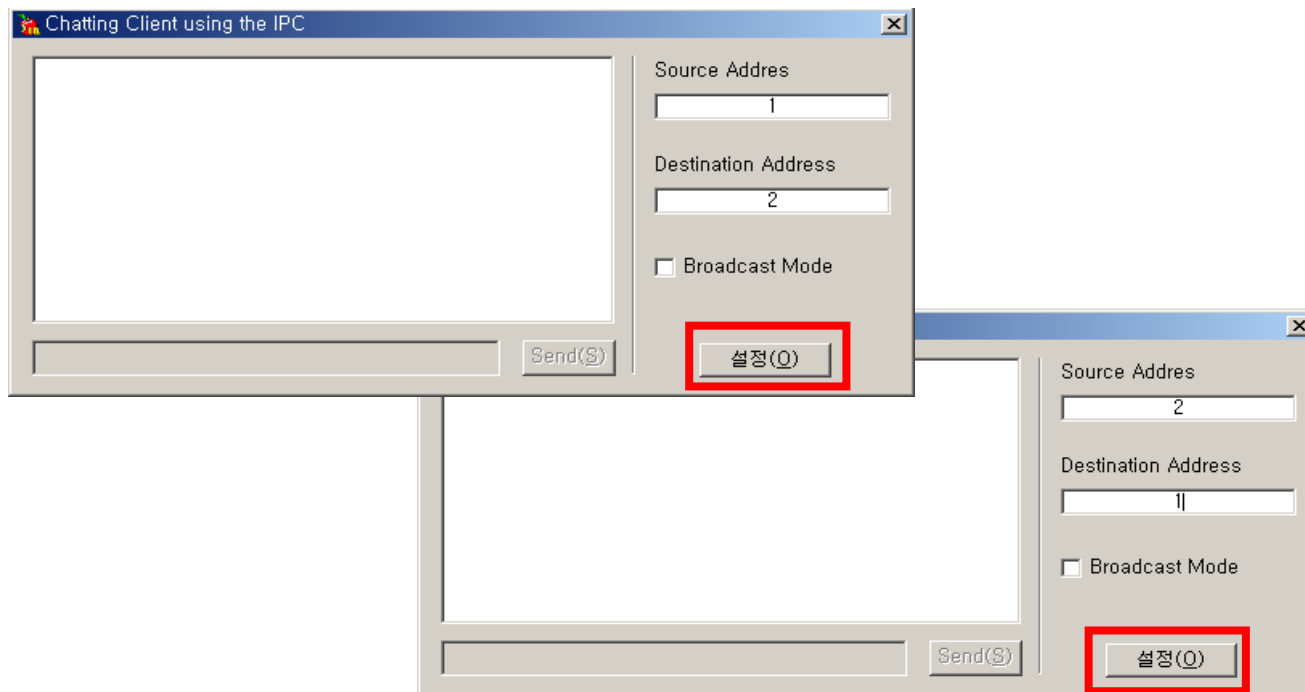
◆ Source & Destination Address 입력





User Interface example

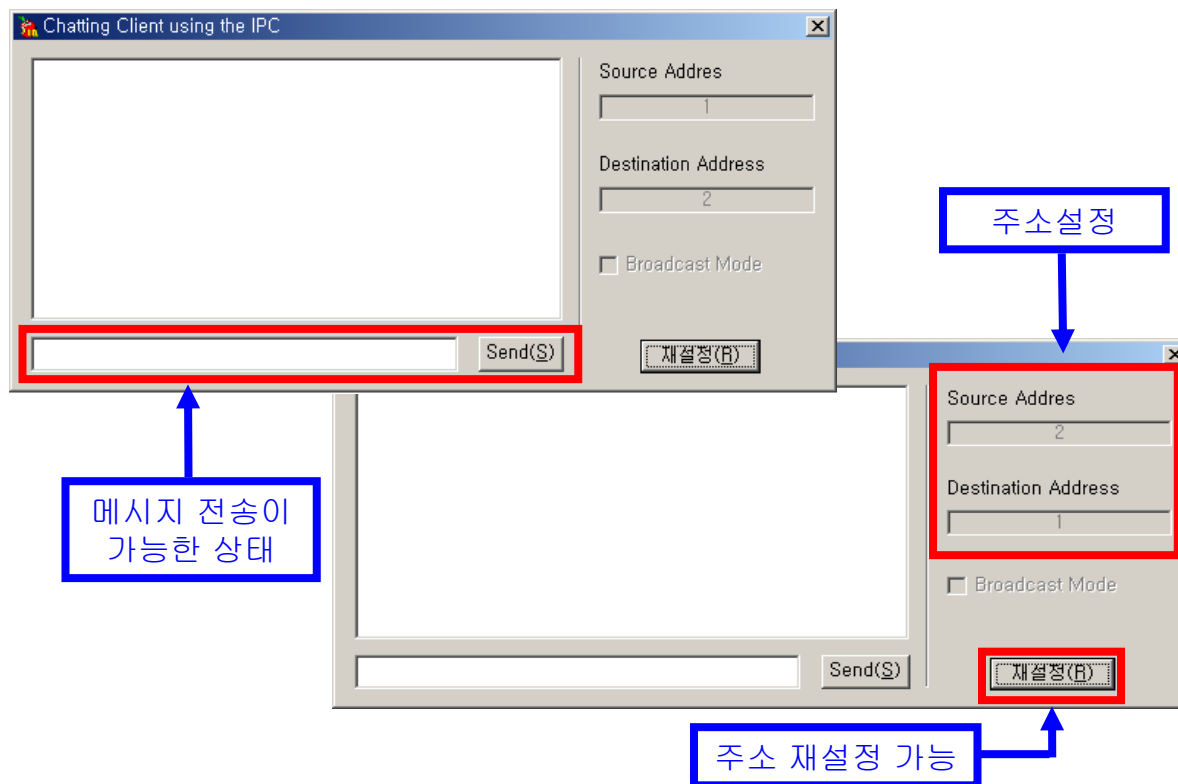
◆ 입력한 주소를 프로그램에 적용





User Interface example

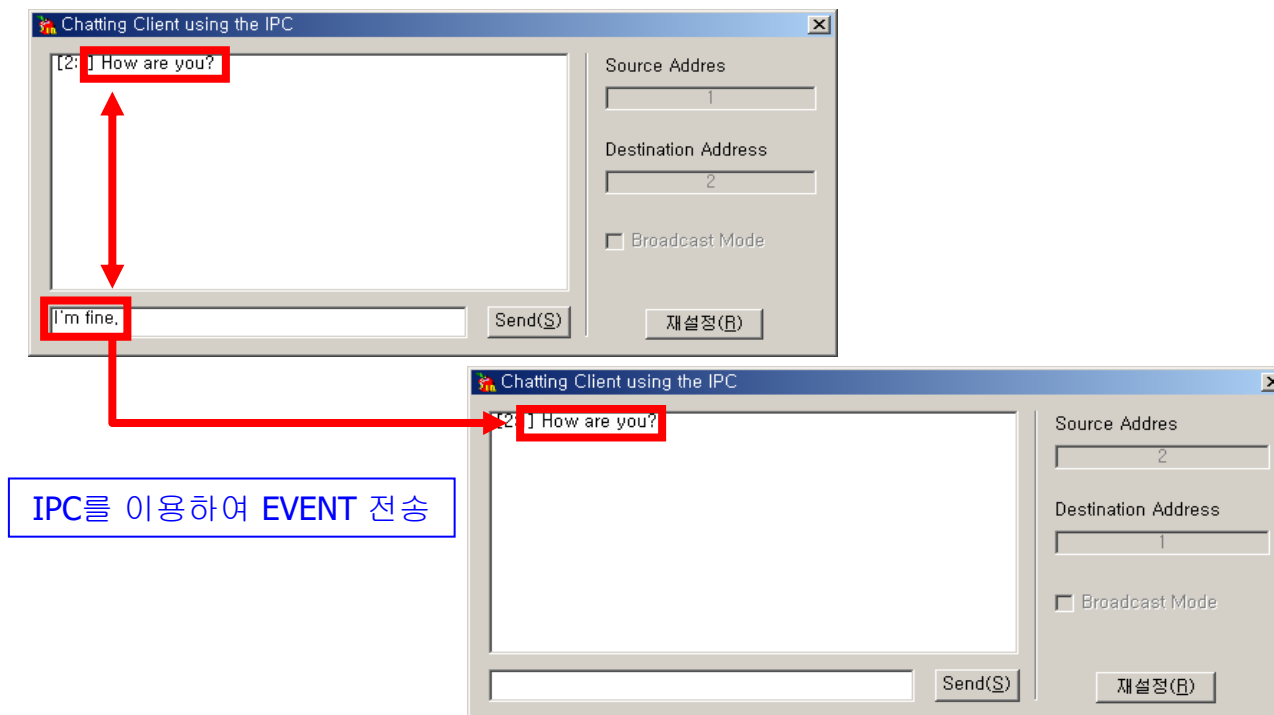
◆ 텍스트 입력 모드





User Interface example

◆ 텍스트 입력 시 결과 화면



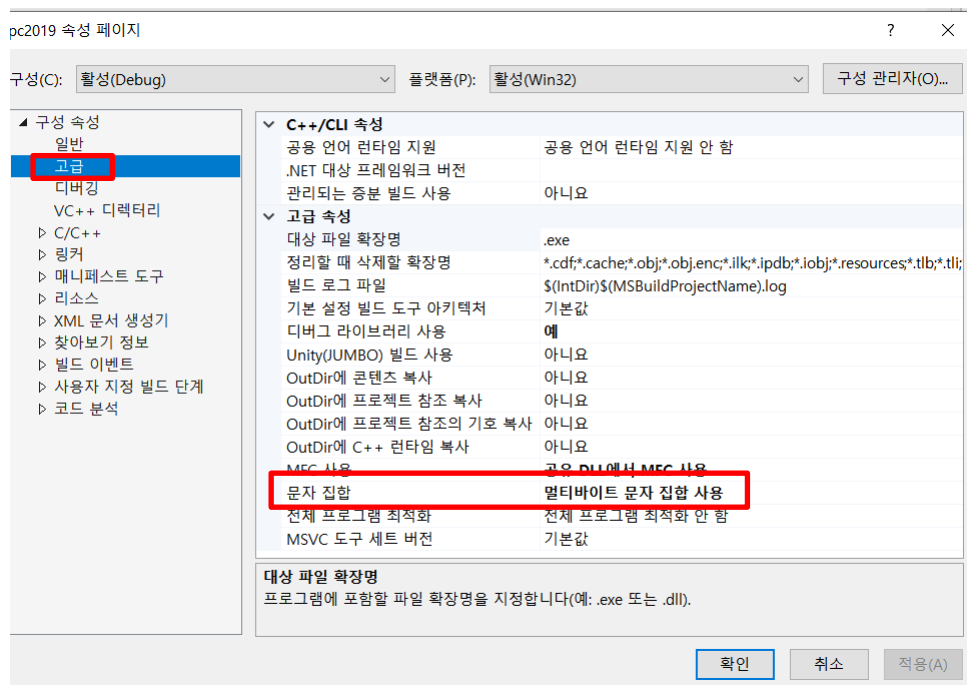


Notice

◆ 참고 사항

❖ Project Setting

◆ 프로젝트 속성 → 고급 → 문자 집합 → 멀티바이트 문자 집합 사용 → 적용





Notice

◆ Link Connect

❖ ipc2019Dlg.cpp 파일

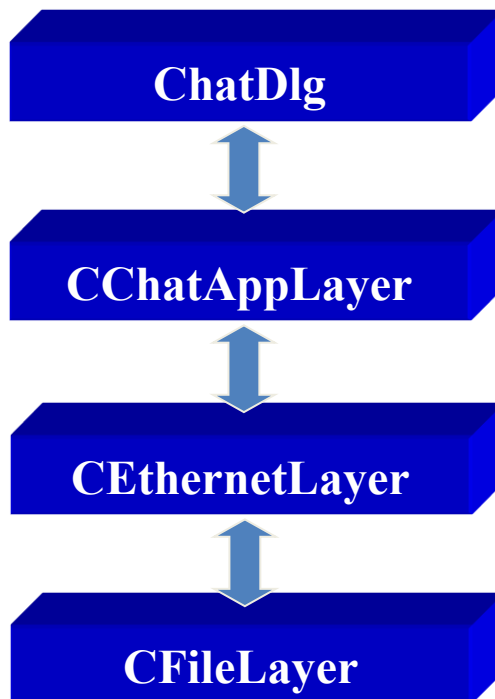
```
// 레이어를 연결한다. (레이어 정보)  
m_LayerMgr.ConnectLayers(??);
```





Notice

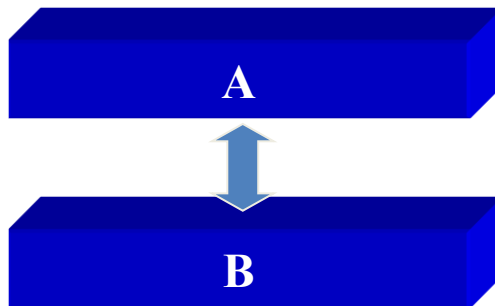
- ◆ Link Connect
 - ❖ Program Hierarchical Architecture



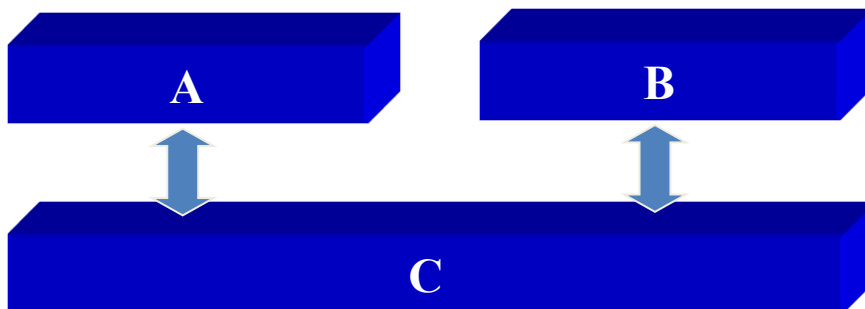


Notice

◆ “B (*A)”



◆ “C (*B *A)”



Q&A

CLab.